INSTITUTO FEDERAL
Mato Grosso
Campus Cuiabá
Cel. Octayde Jorge da Silva

Instituto Federal de Educação, Ciência e Tecnologia de Mato Grosso
Curso: Engenharia da Computação
Disciplina: Transmissão e comunicação de dados
Professor: Ruy Oliveira
Alunos: Emily Weber de Arruda, Felipe Enzo Fukase e Kauan Santos Rondon

Relatório: Detecção de erros em telemetria sob ruído real

Integrante	Matrícula	Responsabilidade principal
Emily Weber de Arruda	2025178440003	TX
Felipe Enzo Fukase	2025178440014	RX
Kauan Santos Rondon	2025178440015	Ruído

Repositório: https://github.com/kauanSR-DEv/telemetria-esp32

Cuiabá - 2026

1 - Introdução e objetivos

Este trabalho apresenta um sistema de telemetria serial entre duas placas ESP32, submetido a ruído físico controlado por uma terceira placa. O protocolo utiliza enquadramento, numeração sequencial e um campo de detecção de erros, configurado para soma modular de 8 bits ou CRC-8 em rodadas distintas. O receptor aceita ou descarta cada quadro com base na checagem e, posteriormente, compara o payload recebido com uma referência para avaliar a integridade dos dados. O objetivo é comparar as duas técnicas sob diferentes modos de ruído, considerando a detecção de corrupção, a perda de quadros, o tempo de processamento e a sobrecarga de transmissão. A análise relaciona o comportamento observado às propriedades de cada algoritmo.

O objetivo geral é desenvolver e avaliar um protocolo de telemetria UART sob ruído físico controlado, comparando a capacidade de detecção de erros da soma modular e do CRC-8. Para isso, o trabalho estabelece os seguintes objetivos específicos.

- Especificar formato, sincronização, sequência, descarte, timeouts e recuperação do protocolo.
- Implementar TX, RX e NOISE com registros que permitam relacionar cada envio às alterações observadas.
- Classificar os quadros nas condições de ruído previstas e calcular a taxa de não detecção.
- Medir tempos de cálculo e verificação, overhead e recuperação após perda.
- Explicar um erro real não detectado por meio dos bytes recebidos, do log de injeção e do cálculo manual.

O TX produz o payload e anexa a checagem. O RX interpreta o quadro e aceita ou descarta os dados. O NOISE acompanha o sinal antes e depois do ponto de injeção. A comunicação avaliada segue do TX para o RX, sem confirmação nem retransmissão. Cada integrante assume uma placa, e a defesa do sistema envolve o entendimento do conjunto.

2 - Fundamentação teórica

A UART realiza comunicação serial assíncrona. A configuração adotada é 115200 baud e 8N1: oito bits de dados, sem paridade e um bit de parada. Incluindo o bit de início, cada byte ocupa dez períodos de bit na linha. A API Hardware Serial permite configurar esses parâmetros e os GPIOs utilizados.

$$Tbit = \frac{1}{115200}\, s \approx 8,681\ \mu s$$

$$Tbyte = 10\, Tbit \approx 86,806\ \mu s$$

O marcador de início permite encontrar um quadro no fluxo de bytes. O campo de comprimento delimita o payload; a sequência permite identificar lacunas e repetições. Uma falha de enquadramento pode impedir a conclusão da checagem, situação classificada como perda, mesmo quando o observador físico registrou bytes na linha. Na técnica SUM, os cinco bytes protegidos são somados ao módulo 256. O TX envia o inverso aditivo dessa soma como CHECK, representado por c nas equações. O RX soma novamente os dados e o CHECK recebido; o quadro é aceito quando o resultado é zero. Alterações cuja soma algébrica seja múltipla de 256 preservam esse resíduo .

$$S = (LEN + SEQH + SEQL + P0 + P1)\ mod\ 256$$

$$c = (-S)\ mod\ 256$$

$$rSUM = (LEN + SEQH + SEQL + P0 + P1 + c)\ mod\ 256$$

O CRC pode ser descrito por operações polinomiais sobre GF(2), em que soma e subtração correspondem ao XOR. A implementação usa registrador de oito bits e polinômio 0x07, sem o termo de maior grau na representação hexadecimal. Os parâmetros coincidem com os do CRC-8/SMBUS .

$$g(x) = x^8 + x^2 + x + 1$$

O TX calcula o CRC dos cinco bytes protegidos. O RX processa esses mesmos campos e o CHECK recebido, esperando resíduo 0x00. O processamento de cada byte pelo CRC começa no bit mais significativo; na transmissão física UART, os bits de dados são enviados do menos significativo para o mais significativo. Essas duas ordens pertencem a etapas distintas. O transistor NPN força o nível baixo. Portanto, um alvo que estava em 1 pode passar a 0; um alvo que já estava em 0 pode permanecer igual. A quantidade sorteada de bits não representa necessariamente a quantidade alterada. O efeito é verificado comparando os bytes observados antes e depois do resistor da linha.

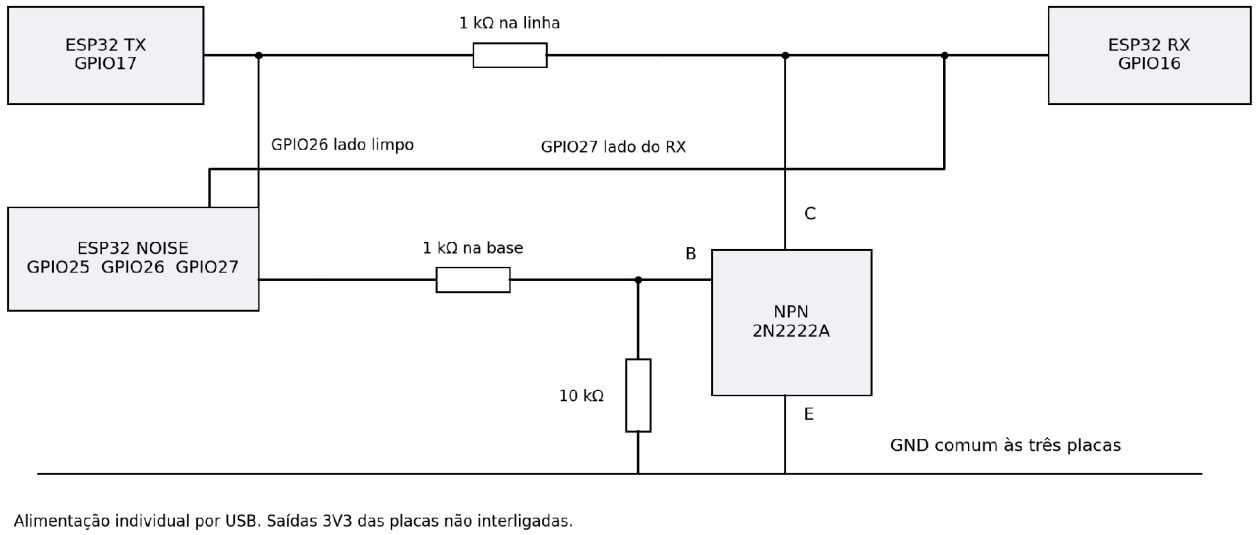
3 - Materiais e montagem

A montagem utiliza três ESP32 clássicos do tipo DevKit V1 - WROOM-32, protoboard, jumpers, um transistor NPN identificado como 2N2222A, dois resistores de 1 kΩ e um resistor de 10 kΩ. As placas são alimentadas individualmente por USB e compartilham o GND. As saídas 3V3 não são interligadas no esquema adotado.

Tabela 1 Conexões do enlace e do injetor

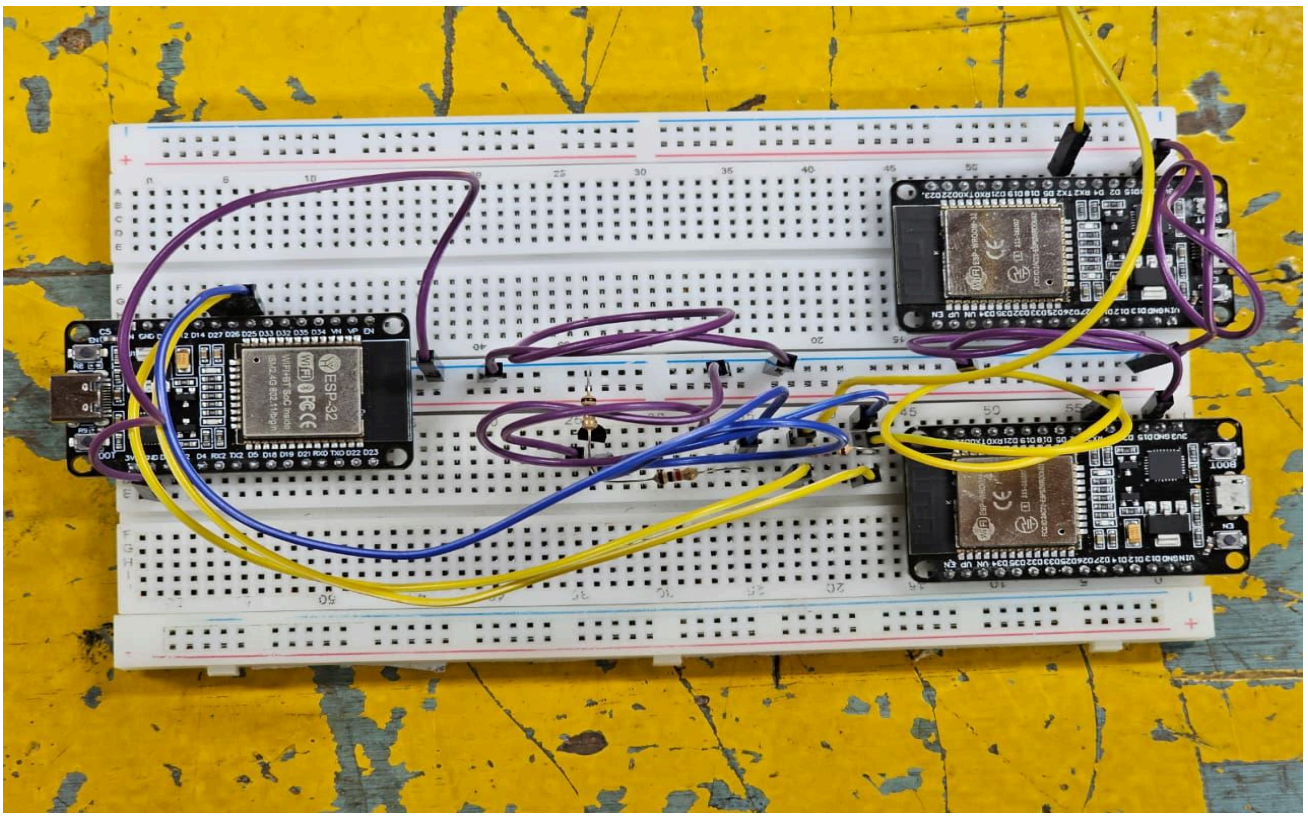
Origem	Destino	Função
TX GPIO17	1 kΩ em série e nó RX GPIO16	Transportar a telemetria
NOISE GPIO26	Linha antes do resistor série	Observar o sinal original
NOISE GPIO27	Nó RX após o resistor série	Observar o sinal perturbado
NOISE GPIO25	1 kΩ até a base do NPN	Acionar a injeção
Base do NPN	10 kΩ até GND	Manter o transistor desligado em repouso
Coletor e emissor do NPN	Nó RX e GND respectivamente	Forçar o nó RX ao nível baixo
GND das três placas	Referência comum	Estabelecer o mesmo nível de referência

Figura 1 Esquema das conexões descritas no código



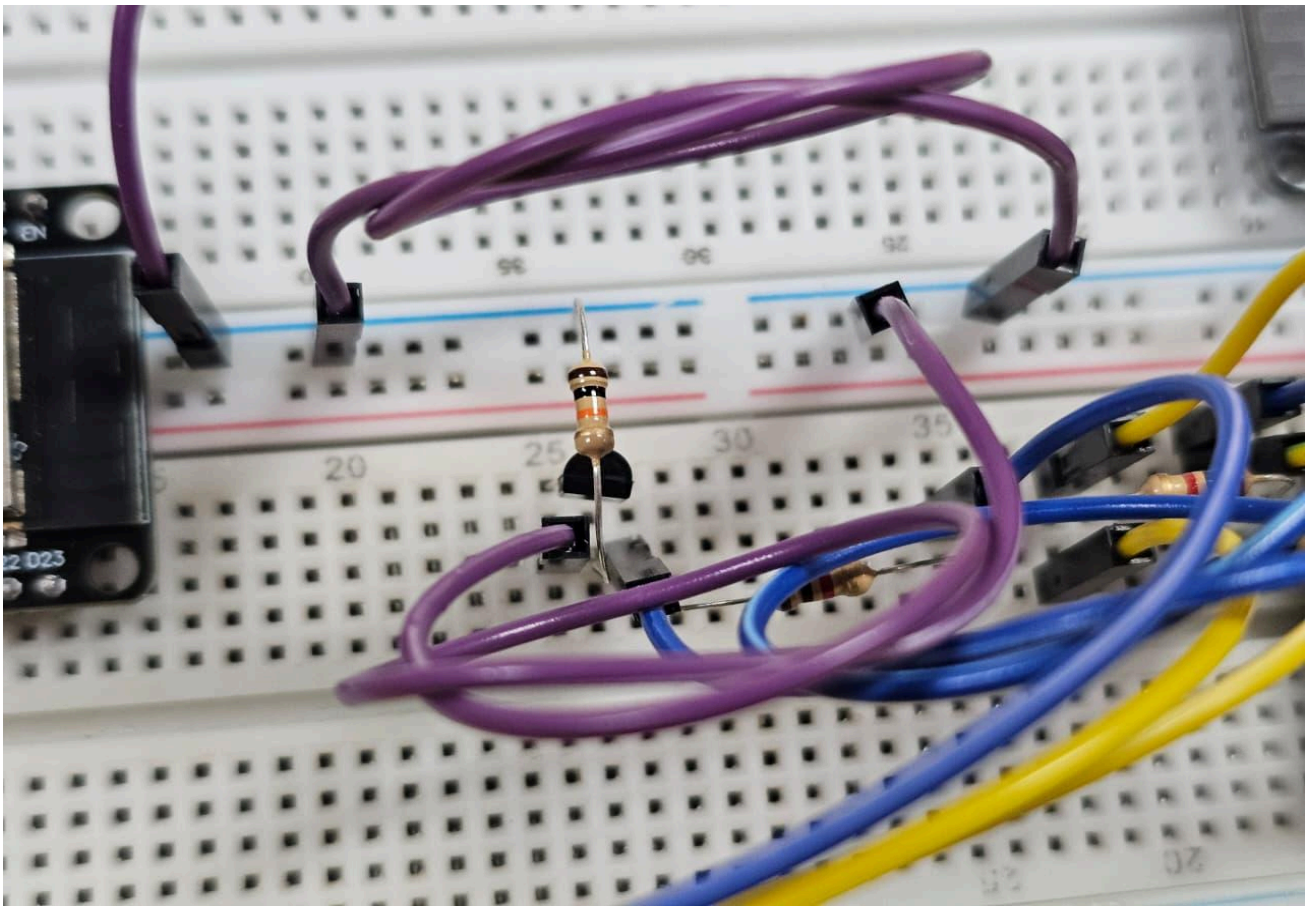
Fonte: Elaborado pelo próprio autor.

Figura 2 Montagem completa com as três placas ESP32



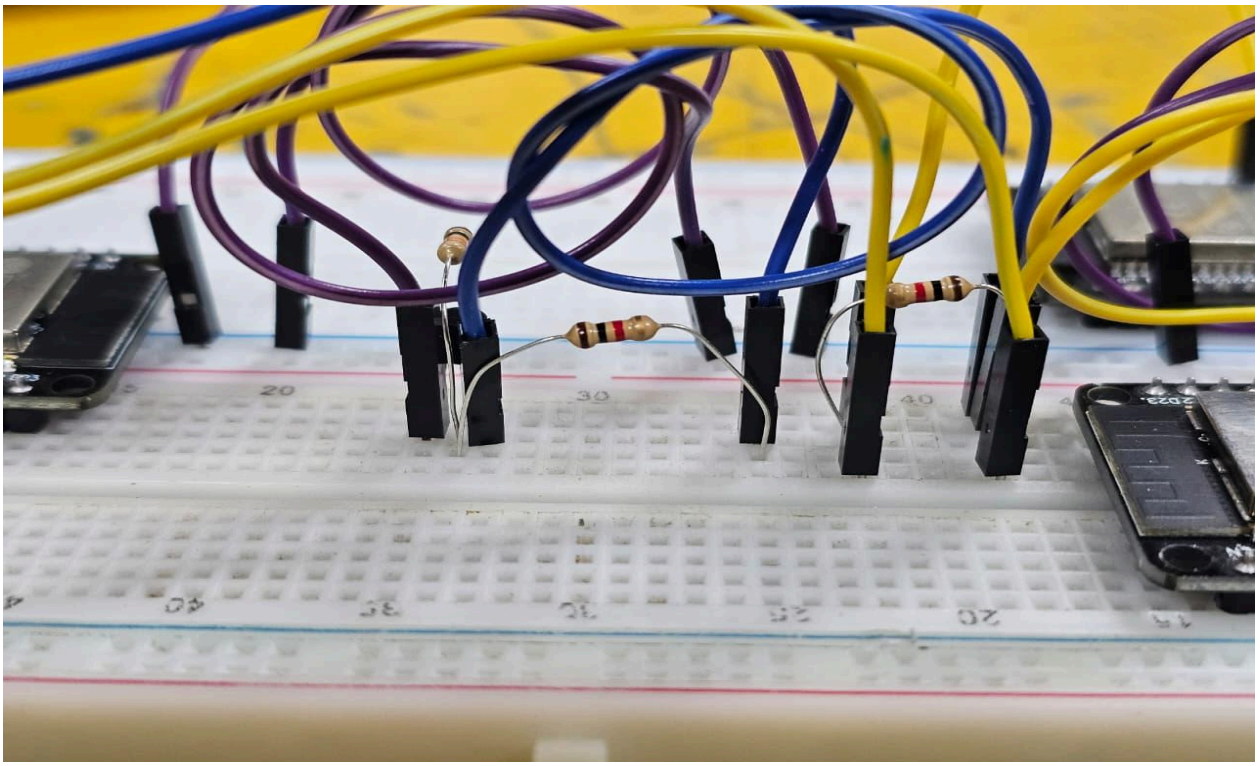
Fonte: Galeria dos autores.

Figura 3 Detalhe do transistor no circuito de injeção



Fonte: Galeria dos autores.

Figura 4 Detalhe dos resistores e conexões do injetor



Fonte: Galeria dos autores.

4 - Especificação do protocolo

O protocolo desenvolvido transmite bytes binários e mantém o mesmo quadro nas duas técnicas. O TX envia até 1000 quadros por sessão, com sequências de 0 a N − 1 e período nominal de 50 ms. O contador representado no quadro possui 16 bits. Os campos de dois bytes utilizam o byte mais significativo primeiro.

Tabela 2 Formato do quadro de telemetria

Posição	Campo	Bytes	Conteúdo	Proteção
0 e 1	SOF	2	A5 5A	Fora da checagem
2	LEN	1	02	Sim
3 e 4	SEQ	2	SEQH e SEQL	Sim
5 e 6	PAYLOAD	2	P0 e P1	Sim
7	CHECK	1	SUM ou CRC-8	Incluído na verificação

Fonte: Elaborado pelo próprio autor.

O quadro ocupa oito bytes. LEN informa os dois bytes de payload; CHECK encerra o quadro. Optou-se pelo uso do campo de comprimento (LEN) em vez de um marcador de fim porque o protocolo transmite dados binários brutos, evitando que a ocorrência acidental de uma sequência de fim dentro do payload ou do CHECK encerre prematuramente o quadro ou exija técnicas de byte stuffing. Durante a leitura de um quadro, uma ocorrência de A5 5A nos dados é tratada como conteúdo. Quando o receptor está buscando início, um marcador acidental pode ser interpretado como candidato e deve passar pelas validações seguintes. Após concluir fisicamente o quadro, o TX espera pelo menos 2 ms e transmite uma referência de sete bytes, com sequência e payload originais. Ela possui marcador próprio e fica fora da região de injeção e da checagem do quadro.

D3 91 SEQH SEQL 02 P0 P1

O RX fixa a decisão antes de consultar o conteúdo desta referência. Em seguida, usa a cópia para verificar se um payload aceito estava correto. A referência não é usada para corrigir dados ou alterar a decisão original. Seu tamanho é excluído do overhead exigido no experimento.

Tabela 3 Parâmetros da checagem

Parâmetro	SUM	CRC-8
Largura e campo CHECK	8 bits e 1 byte	8 bits e 1 byte
Entrada no TX	LEN SEQH SEQL P0 P1	LEN SEQH SEQL P0 P1
Geração	Inverso aditivo módulo 256	Polinômio 0x07
Inicialização	Soma em zero	0x00
Reflexão e XOR final	Não se aplicam	Sem reflexão e XOR final 0x00
Resíduo no RX	0x00 com CHECK	0x00 com CHECK

Fonte: Elaborado pelo próprio autor.

Após concluir a checagem, o RX retoma a busca pelo marcador a partir do byte seguinte a CHECK. Se LEN for diferente de 02 ou ocorrer timeout interno, o candidato é abandonado e a busca recomeça após LEN, reaproveitando os bytes posteriores já recebidos. Um quadro com

checagem reprovada não entrega o payload à aplicação, é registrado como CORROMPIDO e não é contado novamente como PERDIDO .

Para cada quadro aceito, calcula-se d = (sequência recebida − sequência esperada) módulo 65536. Se d = 0, o quadro está em ordem. Se 1 ≤ d ≤ 32767, há d posições saltadas na sequência; nos dois casos, a próxima sequência esperada passa a ser a recebida mais um, módulo 65536. Valores de d a partir de 32768 são tratados como repetição ou chegada fora de ordem, sem alterar a expectativa. A passagem de 65535 para zero é tratada pelo cálculo modular, embora as sessões executadas usem somente as sequências de 0 a 999. A quantidade de posições saltadas não é somada diretamente ao total de PERDIDOS: a auditoria reconcilia os envios e identifica aqueles cuja checagem não foi concluída, sem contar novamente os já classificados como CORROMPIDO. A sequência de um quadro com checagem reprovada não é usada como valor confiável para calcular lacunas .

5 - Implementação de TX, RX e NOISE

O TX gera uma sequência reproduzível de payloads, calcula SUM ou CRC e transmite os quadros pela UART2 no GPIO17. A mesma sequência de dados é utilizada nas duas técnicas, permitindo uma comparação sob condições equivalentes. Os registros CSV incluem os dados enviados e o tempo de cálculo da checagem. O RX recebe os quadros pela UART2 no GPIO16, identifica seu início e verifica a integridade. Resíduo zero resulta em aceitação; caso contrário, o quadro é descartado. A referência é consultada posteriormente para identificar alterações no payload que passaram pela checagem. Quadros incompletos ou com comprimento inválido são abandonados, e o receptor retoma a busca pelo próximo quadro.

O controle de sequência identifica lacunas, repetições e quadros fora de ordem, enquanto a auditoria contabiliza cada envio uma única vez. Cada sessão começa na sequência zero e transmite até 1000 quadros. Erros de UART ou inconsistências na auditoria invalidam a coleta, que deve ser encerrada com END somente após a indicação DONE do TX.

Tabela 4 Limites temporais implementados no RX

Evento	Início da contagem	Limite
Quadro incompleto	SOF completo	2 ms
Espera entre quadros	Último SOF reconhecido	75 ms
Referência incompleta	Início da referência	5 ms
Aviso de TX ausente	RX armado sem primeiro quadro	1 s sem contar perdas

Fonte: Elaborado pelo próprio autor.

A implementação considerada é noise/noise.ino do pacote ESP32_NOISE_v1_12_sincronizado.zip. Os fontes indicam Arduino-ESP32 3.3.12 e alvo ESP32 clássico. O injetor fixa a CPU em 240 MHz e verifica APB em 80 MHz. O GPIO25 controla o NPN, e as UARTs observadoras recebem pelos GPIO26 e GPIO27, sem transmitir para o enlace.

Tabela 5 Modos de ruído implementados

Modo	Plano por quadro	Região elegível
0	Sem pulsos	Nenhum bit
1	Um bit pseudoaleatório	48 bits dos bytes 2 a 7
2	Dois bits distintos pseudoaleatórios	48 bits dos bytes 2 a 7
3	Janela contígua de 3 a 8 bits	Um dos bytes 2 a 7

Fonte: Elaborado pelo próprio autor.

Os alvos do ruído incluem LEN, sequência, payload e CHECK, preservando o SOF, os bits de início e parada da UART e a cópia de depuração. O planejamento ocorre antes da sessão, com semente configurável pelo comando SEED. No modo 3, a janela sorteada pode conter zeros, resultando em menos alterações efetivas que alvos. O marcador A5 5A sincroniza a injeção. Um temporizador de 10 MHz, com resolução de 0,1 µs, inicia os eventos, e o contador de ciclos CCOUNT controla as transições seguintes na mesma interrupção, sem utilizar delayMicroseconds.

A versão 1.12 ajusta a leitura do GPIO26, o escalonamento dos núcleos e a recuperação de bordas atrasadas para reduzir interferências na captura e nos pulsos. A eficácia desses ajustes é verificada nos registros de cada rodada.

Tabela 6 Configuração temporal do NOISE

Parâmetro	Valor
Fase inicial baseada na bancada	−55 ticks ou −5,5 µs
Antecipação do alarme	60 ticks ou 6,0 µs
Atraso máximo admitido do evento	20 ticks ou 2,0 µs
Adiantamento máximo admitido	5 ticks ou 0,5 µs
Tolerância normal do marcador	25 ticks ou 2,5 µs
Borda tardia isolada com confirmação	Até 43 ticks sob regras do detector

Fonte: Elaborado pelo próprio autor.

6 - Metodologia e demonstração experimental

Foram executadas sete condições em 19 de outubro de 2026, com 1000 quadros por condição, semente 1 e NOISE v1.12-sincronizado. Mantiveram-se circuito, baud rate, formato, sequência de payloads e parâmetros do injetor para a comparação entre as técnicas. As rodadas SUM cobriram os modos 0 a 3; as rodadas CRC-8, os modos 1 a 3. Os horários da Tabela 7 seguem o fuso UTC−04:00 .

Cada envio recebe uma única classe: OK quando aceito com payload correto; CORROMPIDO quando a checagem concluída falha; PERDIDO quando a checagem não é concluída; NÃO DETECTADO quando aceito com payload incorreto. Referência ausente ou associação inconsistente tornam o registro inconclusivo e impedem validar a coleta.

Tabela 7 Identificação das sete condições coletadas

Técnica	Modo	Sessão TX	N	Semente	Início em 19/10	Validação
SUM	0	3	1000	1	17:49:27	Válida
SUM	1	4	1000	1	17:50:48	Válida
SUM	2	5	1000	1	17:52:57	Válida
SUM	3	6	1000	1	17:54:41	Válida
CRC-8	1	8	1000	1	18:07:38	Válida
CRC-8	2	9	1000	1	18:13:55	Válida
CRC-8	3	10	1000	1	18:15:46	Válida

Fonte: Elaborado pelo próprio autor.

1. A gravação das três saídas seriais foi iniciada antes dos envios, preservando cabeçalhos, dados, avisos e resumos. Técnica, modo, semente, portas e horário da rodada foram registrados em sessao.json.
2. Com as sessões paradas, selecionou-se SUM ou CRC no TX e no RX e configuraram-se MODE e SEED no NOISE.
3. O início seguiu a ordem RX, NOISE e TX, com START 1000 em cada placa.
4. Esperou-se DONE no TX antes do encerramento do RX com END; o NOISE concluiu sua contagem ou recebeu END após o término confirmado do transmissor.
5. Foram conferidas as quatro classes, a ausência de registros inconclusivos e a integridade da referência. A auditoria cruzou os quadros originais e perturbados do NOISE com os registros do TX e as decisões do RX.

A condição precisa apresentar 1000 envios confirmados. No RX, as classes somam 1000, INCONCLUSIVO e PENDENTE são zero e coleta valida é SIM. No NOISE, esperam-se 1000 registros, SOF e observações correspondentes nas duas UARTs, zero falhas e val ido igual a SIM. Uma contabilidade válida do RX não substitui a validação física do injetor .

O log do NOISE diferencia a máscara planejada alvos_hex do XOR medido xor_hex. Uma rodada com alvos que já eram zero pode ser válida sem mudança nesses bits. Falhas de temporização, alteração fora dos alvos ou mudança da referência tornam a condição inadequada para a comparação final.

Na demonstração ao vivo, o modo de ruído é escolhido sem aviso prévio para o teste de predição. Antes do envio, cada integrante registra quantos dos dez quadros espera em cada classe (OK, CORROMPIDO, PERDIDO e NÃO DETECTADO) e justifica sua estimativa com base nas frequências obtidas nas tabelas experimentais de 1000 quadros, comparando em seguida a previsão ao resultado registrado pelo RX. Ressalta-se que uma amostra reduzida de dez quadros está sujeita a flutuações estatísticas e pode diferir das proporções exatas de 1000 envios sem invalidar o experimento. Os códigos-fonte comentados (tx.ino, rx.ino e noise.ino), a especificação e os dados das coletas estão disponíveis no repositório GitHub indicado na capa. Cada condição experimental está identificada por sua respectiva pasta contendo os registros brutos (tx.txt, rx.txt, noise.txt e sessao.json), a partir dos quais os arquivos CSV e resumos são derivados, reunidos no pacote coletas.zip.

7 - Resultados e desempenho

Os resultados a seguir foram recalculados a partir dos registros integrais enviados em coletas.zip, correspondentes às sete condições da Tabela 7. Cada condição contém 1000 classificações do RX, somando 7000 envios na comparação. Seis condições passaram pela validação sem ressalvas; CRC-8 modo 1 possui uma ressalva de integridade documental descrita adiante .

Tabela 8 Resultados da soma modular

Modo	OK	Corrompido	Perdido	Não detectado	Taxa %
0	1000	0	0	0	N/A
1	616	368	16	0	0,000000
2	384	576	38	2	0,346021
3	174	740	86	0	0,000000

Fonte: Elaborado pelo próprio autor.

Tabela 9 Resultados do CRC de oito bits

Modo	OK	Corrompido	Perdido	Não detectado	Taxa %
1	621	363	16	0	0,000000
2	399	563	38	0	0,000000
3	167	747	86	0	0,000000

Fonte: Elaborado pelo próprio autor.

A taxa de não detecção considera somente CORROMPIDO e NÃO DETECTADO no denominador; os perdidos ficam fora desse cálculo. N/A no baseline indica denominador zero, e não uma taxa estimada de zero. No SUM modo 2, a taxa foi:

$$Taxa\ de\ não\ detecção\ (\%)\ =\ 100 \times \frac{NÃO\ DETECTADO}{CORROMPIDO + NÃO\ DETECTADO}$$

$$100 \times 2 \div (576\ +\ 2)\ =\ 0,346021\%$$

O SUM aceitou dois payloads incorretos no modo 2. O CRC-8 não apresentou erros não detectados nas três condições executadas. As perdas foram iguais entre as técnicas para o mesmo modo: 16, 38 e 86. Os números de quadros OK e corrompidos diferiram porque CHECK também pode ser sorteado pelo injetor e contém valores distintos nas duas técnicas. Os resultados descrevem estas rodadas e não estabelecem imunidade a qualquer padrão de erro.

Na coleta SUM modo 2, o NOISE registrou 2000 alvos e 748 alterações efetivas de bits, com zero falhas. Isso não equivale a 748 quadros corrompidos: dois alvos podem atingir o mesmo quadro, e alvos originalmente em zero não mudam. As duas colisões são analisadas na seção 8.

Ressalva da captura CRC-8 modo 1

Durante a coleta com CRC-8 no modo 1, foi identificado um problema pontual na captura do TX: a linha da sequência 695 apresentou 64 bytes FF nos campos iniciais. Apesar disso, o quadro permaneceu legível e coincidiu com o registro original do NOISE, enquanto o RX preservou as 1000 classificações e o resumo final. O cruzamento dos registros do TX, RX e NOISE confirmou a

validade das classificações, com 1000 registros válidos, sem falhas de injeção ou erros de UART. Como a origem dos bytes FF não foi determinada, a coleta é apresentada com essa ressalva e os arquivos brutos foram preservados. A linha parcial foi excluída das médias de cálculo e intervalo, resultando em 999 amostras de cálculo do TX e 998 intervalos completos; os valores ausentes não foram estimados.

Tabela 10 Tempos médios de cálculo e verificação por quadro

Condição	TX µs	Amostras TX	RX µs	Checagens RX
SUM modo 0	1,0010	1000	3,4920	1000
SUM modo 1	1,0010	1000	3,0020	984
SUM modo 2	1,0010	1000	3,2703	962
SUM modo 3	1,0010	1000	3,2910	914
CRC-8 modo 1	3,0020	999	6,9411	984
CRC-8 modo 2	3,0020	1000	6,0031	962
CRC-8 modo 3	3,0020	1000	6,3085	914

Fonte: Elaborado pelo próprio autor.

O TX mede a execução de makeCheck sobre os cinco bytes protegidos; o RX mede a checagem desses campos e do CHECK recebido, totalizando seis bytes. As impressões seriais ficam fora dos intervalos medidos. Os tempos foram obtidos por quadro, sem usar BENCH para preencher a tabela. A resolução em microssegundos quantiza os valores individuais. O CRC exigiu mais tempo nas rotinas executadas, mantendo o mesmo byte adicional de detecção .

$$Overhead = 8 - 2 = 6\,bytes\,por\,quadro$$

O overhead é de seis bytes: SOF ocupa dois, LEN um, SEQ dois e CHECK um. Para um payload de dois bytes, a razão overhead/payload é 300% e a eficiência dentro do quadro é 25%. A referência de sete bytes e os bits de início e parada da UART ficam fora desse cálculo, conforme o critério do trabalho .

Tabela 11 Intervalos entre quadros registrados pelo RX

Condição	Mínimo ms	Média ms	Máximo ms	Intervalos
SUM modo 0	50,996	51,001011	51,957	999
SUM modo 1	50,998	51,000004	51,003	999
SUM modo 2	50,919	50,999989	51,007	999
SUM modo 3	50,920	50,999986	51,006	999
CRC-8 modo 1	50,990	51,000070	51,007	999
CRC-8 modo 2	50,998	51,000004	51,004	999
CRC-8 modo 3	50,916	50,999990	51,011	999

Fonte: Elaborado pelo próprio autor.

O TX registrou médias próximas de 51,000 ms, embora o período nominal do código seja 50 ms. No RX, o maior intervalo observado foi 51,957 ms, no baseline, e não houve perda entre seus 1000 quadros. O timeout entre quadros de 75 ms supera esse máximo em 23,043 ms. Assim, o limite evita declarar ausência nos intervalos normais medidos e permite iniciar a recuperação

quando o fluxo deixa de chegar. Essa justificativa utiliza medidas de software do próprio experimento.

A 115200 bit/s e 8N1, o quadro de oito bytes ocupa teoricamente 80 períodos de bit, aproximadamente 694,44 µs. O timeout interno de 2 ms oferece margem para essa recepção e limita a espera após um início reconhecido. O timeout da referência é de 5 ms, contado a partir do início de sua leitura; a guarda anterior de 2 ms é uma etapa separada .

Tabela 12 Recuperação até a próxima decisão aceita

Condição	Média ms	Mínimo ms	Máximo ms	Eventos
SUM modo 0	N/A	N/A	N/A	0
SUM modo 1	89,362	51,110	204,112	16
SUM modo 2	121,057	51,109	612,112	35
SUM modo 3	305,168	51,142	1326,187	52
CRC-8 modo 1	80,731	52,011	154,041	16
CRC-8 modo 2	115,226	51,110	408,112	35
CRC-8 modo 3	350,869	51,112	2040,115	49

Fonte: Elaborado pelo próprio autor.

recuperacao_ms mede do início da condição de recuperação até a próxima decisão aceita, conforme a implementação do RX. Não representa o intervalo entre dois envios. Perdas consecutivas ou novos descartes podem prolongar um mesmo episódio; por isso, a quantidade de eventos medidos não é necessariamente igual à quantidade de quadros perdidos. No baseline, N/A indica que não houve evento de recuperação, e não tempo zero .

Os registros completos estão identificados na referência . Os arquivos estão disponíveis no repositório indicado na capa .

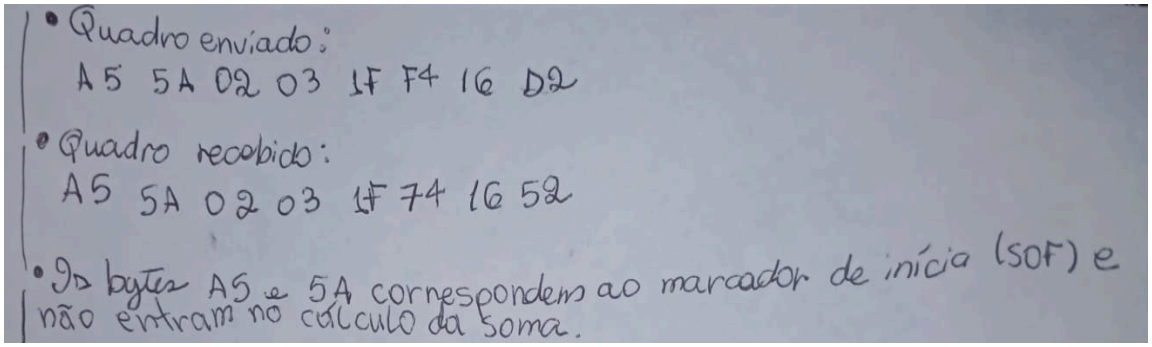
8 - Análise de erros e discussão

Na coleta SUM modo 2, sessão 5, semente 1, ocorreram dois erros não detectados reais, nas sequências 147 e 799. O evento 799 foi conferido nos arquivos tx.txt, rx.txt e noise.txt da pasta 20261019_175257_105181_SUM_M2_seed1. A sequência permaneceu correta, enquanto o primeiro byte do payload e CHECK foram alterados .

Tabela 13 Evento real não detectado na sequência 799

Posição	Campo	TX em hexadecimal	RX em hexadecimal
0 e 1	SOF	A5 5A	A5 5A
2	LEN	02	02
3 e 4	SEQ	03 1F	03 1F
5	P0	F4	74
6	P1	16	16
7	CHECK	D2	52

Figura 5 - Quadros enviado e recebido do evento 799



Fonte : Galeria dos autores

O NOISE registrou os alvos 5.7 e 7.7, com bytes e bits numerados a partir de zero. As duas alterações foram de 1 para 0 no bit de peso 128: F4 passou a 74, e D2 passou a 52. A máscara de alvos e o XOR observado coincidiram em 0000000000800080, sem falhas. Os campos abaixo foram extraídos dos registros do mesmo evento:

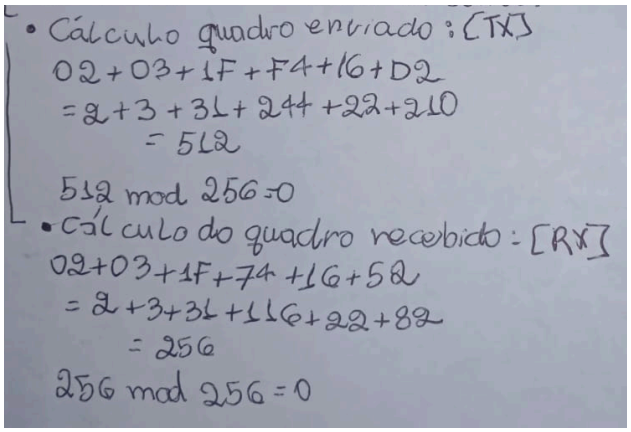
TX: sessao=5; tecnica=SUM; seq=799; t_inicio_us=3301036223; payload_hex=F416; quadro_hex=A55A02031FF416D2.

NOISE: indice=799; seq_original=799; t_sof_us=3407466042; posicoes_byte_bit=5.7;7.7; receptor_hex=A55A02031F741652; alvos_n=2; mudancas_n=2; xor_hex=0000000000800080; falhas_mask=0; qualidade=VALIDO.

RX: seq_original=799; seq_recebida=799; residuo=00; decisao=ACEITO; payload_recebido=7416; payload_referencia=F416; payload_confere=NAO; classe=NAO_DETECTADO.

Os timestamps são locais a cada placa e não representam um relógio global sincronizado. A referência D3 91 03 1F 02 F4 16 foi preservada. Para conferir manualmente SUM, somam-se os seis bytes de LEN até CHECK, excluindo SOF:

Figura 6 - Cálculo manual do erro não detectado pela soma modular (SUM)



Fonte: Galeria dos autores.

A redução total foi 128 + 128 = 256, preservando o resíduo zero módulo 256. O RX aceitou o payload incorreto 7416 e só depois a referência revelou que o original era F416. Esse evento comprova uma vulnerabilidade do próprio protocolo: a alteração conjunta do payload e de CHECK pode conservar a soma. O segundo caso, sequência original 147, alterou os bits 4.7 e 5.7: A5 5A

02 00 93 A5 92 34 passou a A5 5A 02 00 13 25 92 34, também com soma de 512 para 256 e resíduo zero.

Os vetores abaixo foram conferidos por cálculo independente sobre os bytes do protocolo. São verificações matemáticas, sem atribuição de execução física à bancada. O CRC de conferência para a sequência ASCII 123456789 é F4 com os parâmetros adotados, consistente com o catálogo .

Tabela 14 Vetores de verificação dos algoritmos

Entrada protegida em hex	Técnica	CHECK	Resíduo com CHECK
02 00 01 00 C8	SUM	35	00
02 00 01 00 C8	CRC-8	D9	00
02 00 00 80 80	SUM	FE	00
02 00 00 80 80	CRC-8	FB	00

Fonte: Elaborado pelo próprio autor.

No primeiro vetor, os bytes somam 203 e CHECK vale 53, totalizando 256. O resíduo módulo 256 é zero. Esse cálculo ilustra como o campo é gerado e verificado, sem utilizar a referência de depuração. Considere o quadro analítico abaixo, com sequência zero e payload 0x8080. Se o modo 2 selecionar o bit 7 de cada byte do payload e os dois bits forem forçados de 1 para 0, o payload recebido será 0x0000. As posições são 5.7 e 6.7, usando a indexação do NOISE.

TX analítico A5 5A 02 00 00 80 80 FE
RX analítico A5 5A 02 00 00 00 00 FE

$$Soma\,original = 2 + 128 + 128 + 254 = 512$$

$$Soma\,recebida = 2 + 0 + 0 + 254 = 256$$

As duas somas deixam resíduo zero módulo 256. A retirada de dois pesos 128 reduz o total em 256, preservando a checagem. O RX aceitaria o payload incorreto. Aplicando o mesmo padrão ao vetor CRC-8 com CHECK FB, o resíduo passa a 3F, e a checagem reprova o quadro. O evento físico apresentado anteriormente demonstra o erro não detectado com os três registros do mesmo envio. O exemplo analítico complementa essa evidência ao mostrar que duas alterações no payload também podem preservar a soma modular.

O baseline SUM com 1000 quadros OK mostra operação correta do enlace na condição registrada. Nos ensaios com ruído, parte dos quadros continua OK porque um alvo sorteado pode já estar em zero. A probabilidade de alteração efetiva depende dos bytes enviados, inclusive do CHECK, que difere entre SUM e CRC. Por isso, a mesma semente e os mesmos payloads não garantem o mesmo número de mudanças nas duas técnicas .

A escolha de alvos em LEN explica a possibilidade de perda por comprimento inválido. No caso de LEN igual a 02, a retirada de seu único bit 1 produz 00. O receptor abandona o candidato antes de concluir a checagem. Essa perda é consequência do enquadramento adotado e precisa ser distinguida de uma reprovação matemática do SUM ou CRC .

A soma modular é sensível a uma alteração efetiva isolada em um byte protegido, porém admite colisões quando alterações combinadas preservam a soma módulo 256. O padrão de dois bits mais significativos apresentado anteriormente é realizável por duas transições de 1 para 0 e explica por que o modo 2 merece atenção especial. Os registros do evento 799 demonstram esse mecanismo com alterações em payload e CHECK.

No modo 3, o plano restringe a janela a um único byte. Com alterações apenas de 1 para 0, uma mudança efetiva em um único byte protegido reduz sua contribuição por um valor entre 1 e 255; assim, não preserva a soma módulo 256. Uma alteração de LEN pode causar perda antes da

checagem. Essa dedução vale sob as regras do plano e sem alterações extras; uma rajada distribuída por vários bytes constitui outra condição experimental. Para o CRC de grau oito, uma rajada de extensão até oito bits no fluxo protegido, mantido o enquadramento, possui polinômio de erro não nulo de grau inferior a oito após retirar o deslocamento. Como g(x) tem termo constante 1, esse erro não é múltiplo de g(x) e produz resíduo diferente de zero. A dedução explica a detecção dessas rajadas; ela não garante detecção de todos os padrões possíveis.

Vulnerabilidade analítica do CRC-8: padrão não detectado

O CRC-8 também admite padrões de erro que preservam a checagem. Na representação polinomial sobre GF(2), o quadro recebido é dado por:

$$\mathbf{R(x) = T(x) \oplus E(x)}$$

Nessa expressão, T(x) representa os bits transmitidos nos campos protegidos e no CHECK, E(x) representa as alterações e ⊕ indica a operação XOR. O polinômio gerador adotado é:

$$\mathbf{g(x) = x^8 + x^2 + x + 1}$$

Sua representação completa é 0x107; no código, utiliza-se 0x07, omitindo o termo de maior grau. Como o quadro válido possui o resto zero, um erro não é detectado quando E(x) é múltiplo de g(x). Assim:

$$\mathbf{R(x) \bmod g(x) = [T(x) \bmod g(x)] \oplus [E(x) \bmod g(x)] = 0}$$

A operação “mod” representa o resto da divisão polinomial sobre GF(2).

Considere o exemplo analítico abaixo, com sequência zero e payload original 0x8187. O CHECK calculado é 0xFB.

Elemento	Bytes em hexadecimal
Quadro transmitido	A5 5A 02 00 00 81 87 FB
Máscara de erro	00 00 00 00 00 01 07 00
Quadro recebido	A5 5A 02 00 00 80 80 FB

A máscara altera quatro bits do payload: a posição 5.0 e as posições 6.2, 6.1 e 6.0, com bytes e bits numerados a partir de zero. Todos esses bits estavam em 1 e passam a 0, transformando 81 em 80 e 87 em 80, em hexadecimal. Essas transições são compatíveis com a atuação do transistor NPN.

Considerando os seis bytes de LEN até CHECK, a máscara é 00 00 00 01 07 00, correspondente ao polinômio:

$$\mathbf{E(x) = x^{16} + x^{10} + x^9 + x^8 = x^8 \cdot g(x)}$$

Como E(x) é múltiplo do polinômio gerador, o resto da divisão é zero:

E(x) mod g(x) = 0

A verificação dos seis bytes recebidos também resulta em resíduo zero:

CRC-8 de 02 00 00 80 80 FB = 00

Assim, o RX aceitaria o payload incorreto 0x8080, embora o original fosse 0x8187. A divergência seria identificada posteriormente pela referência de depuração. Esse exemplo é uma demonstração analítica, não um evento observado nas coletas. O padrão apresentado exige quatro alterações distribuídas entre dois bytes: os modos 1 e 2 selecionam, respectivamente, um e dois bits por quadro, enquanto o modo 3 restringe a rajada a um único byte. Portanto, os modos implementados não geram essa máscara planejada. A ausência de erros não detectados nas rodadas CRC-8 não significa que o algoritmo detecte todos os padrões possíveis.

O injetor representa perturbações que baixam o nível lógico, e não todos os tipos de ruído de um canal real. A aferição do efeito usa duas UARTs e timestamps de software, sem capturas de osciloscópio ou analisador lógico. A fase inicial provém de calibração anterior da bancada e não é uma constante universal. Há uma rodada por condição, com semente 1; as frequências se limitam às amostras executadas. A ressalva documental de CRC-8 modo 1, descrita na seção 7, permanece explícita . As sete condições possuem 1000 classificações cada, com a decisão do RX separada da referência posterior. O caso real, os intervalos e os eventos de recuperação permitem relacionar os totais à implementação. No CRC-8 não foram observados erros não detectados nos modos 1, 2 e 3 executados; isso não garante proteção contra todos os padrões possíveis .

9 - Conclusão

O enlace implementado reuniu enquadramento, sequência, detecção de erros e auditoria posterior do payload entre duas ESP32, com perturbação física controlada por uma terceira placa. Foram concluídas as sete condições de 1000 quadros com a mesma configuração do injetor e semente 1. Os registros permitiram conferir as decisões do RX separadamente da referência de depuração.

O baseline SUM apresentou 1000 quadros corretos. No modo 2, a soma modular deixou passar dois payloads incorretos, com taxa de não detecção de 0,346021% entre os 576 corrompidos e os dois não detectados. Os bytes dos eventos reais demonstraram uma redução total de 256 na soma, preservando o resíduo módulo 256. O CRC-8 não apresentou erros não detectados nos modos 1, 2 e 3 executados; essa observação se limita aos padrões e às amostras coletadas.

As duas técnicas utilizaram o mesmo overhead de seis bytes e apresentaram iguais totais de perda por modo, evidenciando a influência do enquadramento. O CRC exigiu mais tempo de cálculo e verificação nas rotinas medidas, enquanto a soma apresentou menor custo e uma colisão real documentada. Os intervalos próximos de 51 ms fundamentaram o timeout de 75 ms, e os eventos de recuperação foram registrados até a próxima decisão aceita. A limitação pontual da captura CRC-8 modo 1 foi documentada e suas medidas utilizaram somente as amostras preservadas. Dessa forma, a comparação relaciona capacidade de detecção, perdas, custo de processamento e limites experimentais, com os arquivos brutos mantidos para conferência.

10 - Referências

ESPRESSIF SYSTEMS. Serial UART. Arduino ESP32 documentation. Fonte : https://docs.espressif.com/projects/arduino-esp32/en/latest/api/serial.html

ESPRESSIF SYSTEMS. ESP Timer High Resolution Timer. ESP-IDF Programming Guide. Fonte : https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/esp_timer.html

COOK, Greg. Catalogue of parametrised CRC algorithms (CRC-8/SMBUS). Fonte: [https://reveng.sourceforge.io/crc-catalogue/1-15.htm#crc.cat.crc-8-smbus](https://reveng.sourceforge.io/crc-catalogu /1-15.htm#crc.cat.crc-8-smbus)